

Table of contents

Série 6 : Résolution SAT et Modélisation	1
Méthodes	1
SAT Solving – Algorithme DPLL	1
Transformation en Forme Normale Conjonctive (CNF)	2
Modélisation en Logique Propositionnelle	2
Preuve par Réfutation (Validité d'un raisonnement)	2
Propriété des Ensembles de Clauses Insatisfiables	2
Rappel	3
Exemples Illustratifs	3
Exemple 1 : Transformation en CNF	3
Exemple 2 : Application de DPLL	4
Exemple 3 : Modélisation et Preuve par Réfutation	5
Exemple 4 : Propriété des Ensembles Insatisfiables	5
Synthèse	6

Série 6 : Résolution SAT et Modélisation

Méthodes

SAT Solving – Algorithme DPLL

L'algorithme DPLL (Davis–Putnam–Logemann–Loveland) est une méthode de recherche récursive pour déterminer la satisfiabilité d'une formule propositionnelle sous forme normale conjonctive (CNF).

Il repose sur :

- **Propagation des clauses unitaires** : si une clause ne contient qu'un seul littéral, on l'assigne de façon à satisfaire la clause.
- **Élimination des littéraux purs** : si un littéral n'apparaît que sous une seule polarité dans l'ensemble des clauses, on lui assigne la valeur qui satisfait toutes les clauses le contenant.
- **Choix de variable (splitting)** : en l'absence de clause unitaire ou de littéral pur, on choisit une variable et on explore récursivement les deux branches (variable vraie, puis fausse).
- **Backtracking** : lorsqu'une branche mène à une contradiction (clause vide), on revient en arrière et on essaie l'assignation opposée.

Transformation en Forme Normale Conjonctive (CNF)

Pour appliquer DPLL, les formules doivent être converties en CNF, c'est-à-dire en une conjonction de clauses, chaque clause étant une disjonction de littéraux.

Les étapes typiques sont :

1. Élimination des implications : $A \Rightarrow B$ devient $\neg A \vee B$.
2. Application des lois de De Morgan : $\neg(A \wedge B)$ devient $\neg A \vee \neg B$; $\neg(A \vee B)$ devient $\neg A \wedge \neg B$.
3. Distribution de $\vee$ sur $\wedge$: $A \vee (B \wedge C)$ devient $(A \vee B) \wedge (A \vee C)$.

Modélisation en Logique Propositionnelle

Traduction d'un problème décrit en langage naturel en une formule propositionnelle :

- Identifier les propositions atomiques (ex. : "Alice mange des bonbons").
- Formaliser les contraintes du problème à l'aide des connecteurs logiques ($\wedge$, $\vee$, $\Rightarrow$, $\neg$).
- Combiner les contraintes en une seule formule à analyser.

Preuve par Réfutation (Validité d'un raisonnement)

Pour montrer qu'un raisonnement "Si prémisses alors conclusion" est valide, on prouve que la formule prémisses $\Rightarrow$ conclusion est valide (toujours vraie).

Une méthode efficace :

1. Nier la formule globale : $\neg(\text{prémisses} \Rightarrow \text{conclusion}) \equiv \text{prémisses} \wedge \neg\text{conclusion}$.
2. Mettre cette négation en CNF.
3. Appliquer un solveur SAT (DPLL) : si la négation est **insatisfiable**, alors la formule originale est **valide**.

Propriété des Ensembles de Clauses Insatisfiables

Un ensemble fini de clauses (sans clause vide) qui est insatisfiable contient nécessairement :

- Au moins **une clause positive** (clause dont tous les littéraux sont positifs).
- Au moins **une clause négative** (clause dont tous les littéraux sont négatifs).

La preuve se fait par contradiction : si toutes les clauses sont non-positives (resp. non-négatives), on peut construire une interprétation qui satisfait l'ensemble.

Rappel

- **Lois de De Morgan :**

$$\neg(A \wedge B) \equiv \neg A \vee \neg B$$

$$\neg(A \vee B) \equiv \neg A \wedge \neg B$$

- **Élimination de l'implication :**

$$A \Rightarrow B \equiv \neg A \vee B$$

- **Distributivité de $\vee$ sur $\wedge$:**

$$A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$$

- **Forme Normale Conjonctive (CNF) :** conjonction ($\wedge$) de clauses, chaque clause étant une disjonction ($\vee$) de littéraux.
- **Validité :** une formule est valide si elle est vraie dans toute interprétation. Pour prouver que ψ est valide, on montre que $\neg\psi$ est insatisfiable.
- **DPLL :** algorithme de recherche complet pour SAT. Il utilise la propagation unitaire, l'élimination des littéraux purs et le splitting.

Exemples Illustratifs

Exemple 1 : Transformation en CNF

Contexte :

On considère la formule $\phi = (p_1 \wedge q_1) \vee (p_2 \wedge q_2)$. On veut la transformer en CNF pour compter le nombre de clauses.

Application :

On applique la distributivité de $\vee$ sur $\wedge$:

$$\begin{aligned}\phi &= (p_1 \wedge q_1) \vee (p_2 \wedge q_2) \\ &\equiv (p_1 \vee p_2) \wedge (p_1 \vee q_2) \wedge (q_1 \vee p_2) \wedge (q_1 \vee q_2)\end{aligned}$$

On obtient une CNF avec **4 clauses**.

De manière générale, pour $\phi = (p_1 \wedge q_1) \vee \dots \vee (p_n \wedge q_n)$, la CNF contient 2^n clauses (chaque clause est une combinaison choisissant p_i ou q_i pour chaque i).

Exemple 2 : Application de DPLL

Contexte :

On veut savoir si la formule suivante est satisfiable :

$$\phi = (\neg p \vee q \vee r) \wedge (s \vee p \vee r) \wedge (\neg q \vee \neg r) \wedge (\neg p \vee s) \wedge (\neg q \vee r) \wedge (q \vee \neg s \vee r)$$

Application :

Aucune clause unitaire ni littéral pur évident. On choisit la variable q pour le splitting.

- **Branche $q = \text{Vrai}$** : on substitue q par $\top$ (on retire les clauses contenant q et on retire $\neg q$ des clauses). On obtient :

$$\phi[q] = (s \vee p \vee r) \wedge \neg r \wedge (\neg p \vee s) \wedge r$$

La clause unitaire $\neg r$ impose $r = \text{Faux}$. En substituant $r = \text{Faux}$:

$$\phi[q, \neg r] = (s \vee p) \wedge (\neg p \vee s) \wedge \text{Faux} \quad (\text{car la clause } r \text{ devient fausse})$$

On obtient une contradiction, donc cette branche échoue.

- **Branche $q = \text{Faux}$** : on substitue $\neg q$ par $\top$. On obtient :

$$\phi[\neg q] = (\neg p \vee r) \wedge (s \vee p \vee r) \wedge (\neg p \vee s) \wedge (\neg s \vee r)$$

On choisit $p = \text{Faux}$ ($\neg p$). Alors :

$$\phi[\neg q, \neg p] = (s \vee r) \wedge (\neg s \vee r)$$

On choisit $s = \text{Vrai}$:

$$\phi[\neg q, \neg p, s] = r$$

Clause unitaire : $r = \text{Vrai}$.

Une solution est donc : $q = \text{Faux}, p = \text{Faux}, s = \text{Vrai}, r = \text{Vrai}$. La formule est **satisfiable**.

Exemple 3 : Modélisation et Preuve par Réfutation

Contexte :

Problème des bonbons : modéliser les contraintes et vérifier si le raisonnement “Donc les trois ont mangé des bonbons” est correct.

Application :

Atomes :

- A : Alice mange des bonbons.
- B : Bob mange des bonbons.
- C : Carole mange des bonbons.

Contraintes :

1. $S_1 : A \vee B \vee C$ (au moins un a mangé).
2. $S_2 : A \Rightarrow B$ (si Alice mange, alors Bob aussi).
3. $S_3 : \neg A \Rightarrow \neg B$ (si Alice ne mange pas, alors Bob non plus).
4. $S_4 : (A \wedge B) \Rightarrow C$ (si Alice et Bob mangent, alors Carole aussi).
5. $S_5 : (\neg A \wedge \neg B) \Rightarrow \neg C$ (si ni Alice ni Bob ne mangent, alors Carole non plus).

Conclusion : $S_6 : A \wedge B \wedge C$.

On veut vérifier si $(S_1 \wedge S_2 \wedge S_3 \wedge S_4 \wedge S_5) \Rightarrow S_6$ est valide.

On procède par réfutation : on considère la négation

$$\neg\psi = S_1 \wedge S_2 \wedge S_3 \wedge S_4 \wedge S_5 \wedge \neg S_6$$

avec $\neg S_6 = \neg A \vee \neg B \vee \neg C$.

On applique DPLL sur $\neg\psi$ (après mise en CNF, non détaillée ici).

Le calcul montre que $\neg\psi$ est **insatisfiable** (les deux branches mènent à une contradiction).

Donc ψ est valide : le raisonnement est correct.

Exemple 4 : Propriété des Ensembles Insatisfiables

Contexte :

Montrer que tout ensemble insatisfiable E de clauses (sans clause vide) contient au moins une clause positive et une clause négative.

Application :

Preuve par contradiction.

- Supposons E sans clause positive. Alors chaque clause contient au moins un littéral négatif.
Si on assigne **Faux** à toutes les variables, chaque clause contient un littéral $\neg x$ qui devient vrai (car x est faux), donc toutes les clauses sont satisfaites. Contradiction avec l'insatisfiabilité.
- Supposons E sans clause négative. Alors chaque clause contient au moins un littéral positif.
Si on assigne **Vrai** à toutes les variables, chaque clause contient un littéral x qui devient vrai, donc toutes les clauses sont satisfaites. Contradiction.

Donc E doit contenir à la fois une clause positive et une clause négative.

Synthèse

Cette série d'exercices illustre les techniques fondamentales de la logique propositionnelle pour la vérification :

- La transformation en CNF permet d'appliquer des algorithmes comme DPLL.
- DPLL est un algorithme efficace pour résoudre le problème SAT.
- La modélisation permet de traduire un problème concret en formules logiques.
- La preuve par réfutation permet de vérifier la validité d'un raisonnement.
- Une propriété structurelle des ensembles insatisfiables (présence de clauses positives et négatives) est utile pour des preuves théoriques.

Ces méthodes sont essentielles pour la vérification formelle de systèmes, où l'on doit souvent raisonner sur des contraintes logiques complexes.